

Polymorphism in Python

A Comprehensive Guide

Slide 1: Introduction

What is Polymorphism?

- Polymorphism means "**many forms**" — the same interface can work with different types
- A single function or method can behave differently based on the object it's called on
- Python achieves polymorphism through **duck typing**, **method overriding**, and **operator overloading**
- "If it walks like a duck and quacks like a duck, it's a duck."

```
def make_sound(animal):  
    print(animal.speak())    # Works for any object with .speak()
```

Slide 2: Why Polymorphism Matters

Benefits of Polymorphism:

- **Flexibility** — Write code that works with many types without modification
- **Extensibility** — Add new types without changing existing code
- **Maintainability** — Reduces conditional logic (if/else chains)
- **Abstraction** — Focus on what objects DO, not what they ARE

```
Without polymorphism:           With polymorphism:  
if type(x) == Dog:              x.speak()    # Works for any type  
    x.bark()  
elif type(x) == Cat:             
    x.meow()
```

Slide 3: Duck Typing — Python's Polymorphism

"If it walks like a duck..."

```
class Duck:  
    def quack(self):  
        return "Quack!"  
  
class Person:  
    def quack(self):  
        return "I'm quacking like a duck!"
```

```
def make_it_quack(thing):
    print(thing.quack())

make_it_quack(Duck())      # Quack!
make_it_quack(Person())   # I'm quacking like a duck!
```

- No inheritance required — any object with the matching method works
- Python checks for the **method's existence** at runtime, not the type

Slide 4: Duck Typing vs Static Typing

Aspect	Duck Typing (Python)	Static Typing (Java/C++)
Type check	Runtime	Compile-time
Flexibility	High — any object with matching methods works	Low — must share type hierarchy
Errors caught	At runtime (AttributeError)	At compile time
Code verbosity	Low — no type declarations needed	Higher — interfaces/abstract classes

```
# Python: works with any iterable
def total(items):
    return sum(items)

total([1, 2, 3])      # List - 6
total((1, 2, 3))      # Tuple - 6
total(range(1, 4))    # Range - 6
```

Slide 5: Polymorphism Through Inheritance

Method overriding enables polymorphic behavior:

```
class Animal:
    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"
```

```
# Polymorphic call – same method name, different behavior
animals = [Dog(), Cat(), Dog()]
for a in animals:
    print(a.speak())
# Woof!
# Meow!
# Woof!
```

Slide 6: Polymorphism with Abstract Base Classes

Enforcing polymorphic interfaces:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

    def perimeter(self):
        return 2 * 3.14159 * self.radius

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h

    def area(self):
        return self.w * self.h

    def perimeter(self):
        return 2 * (self.w + self.h)
```

Slide 7: Polymorphic Function with ABC

```
def print_shape_info(shape):
    print(f"Area: {shape.area():.2f}")
```

```

    print(f"Perimeter: {shape.perimeter():.2f}")

shapes = [Circle(5), Rectangle(4, 6)]

for s in shapes:
    print_shape_info(s)

# Circle: Area=78.54, Perimeter=31.42
# Rectangle: Area=24.00, Perimeter=20.00

```

- `print_shape_info()` works with any Shape subclass
- Adding a new shape (Triangle, Square) requires zero changes to this function
- This is the **Open/Closed Principle** in action

Slide 8: Polymorphism with Built-in Functions

Python's built-in functions leverage polymorphism:

```

# len() works on any object with __len__
print(len("Hello"))           # 5 - string
print(len([1, 2, 3]))         # 3 - list
print(len({"a": 1}))          # 1 - dict

# iter() works on any object with __iter__ or __getitem__
for x in "ABC":               # Strings are iterable
    print(x)

# + works on any object with __add__
print(3 + 4)                  # 7 - int
print("Hello " + "World")     # Hello World - str
print([1] + [2, 3])           # [1, 2, 3] - list

```

Slide 9: Operator Overloading — Custom Polymorphism

Define how operators work with your classes:

```

class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)

```

```
def __repr__(self):
    return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)      # Vector(4, 6) – uses __add__
print(v1 * 3)       # Vector(3, 6) – uses __mul__
```

Slide 10: Common Dunder Methods for Operators

Category	Methods	Operators
Arithmetic	<code>__add__</code> , <code>__sub__</code> , <code>__mul__</code> , <code>__truediv__</code>	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>
Comparison	<code>__eq__</code> , <code>__lt__</code> , <code>__gt__</code> , <code>__le__</code> , <code>__ge__</code>	<code>==</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>
Unary	<code>__neg__</code> , <code>__abs__</code> , <code>__invert__</code>	<code>-x</code> , <code>abs(x)</code> , <code>~x</code>
Containment	<code>__contains__</code>	<code>in</code> operator
Indexing	<code>__getitem__</code> , <code>__setitem__</code>	<code>obj[key]</code>

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __eq__(self, other):
        return self.amount == other.amount
```

Slide 11: The `__str__` and `__repr__` Duality

Polymorphic string representation:

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"
        # For developers: unambiguous, should recreate object

    def __str__(self):
        return f"({self.x}, {self.y})"
        # For users: readable, informal
```

```
p = Point(3, 4)
print(str(p))      # (3, 4)          - calls __str__
print(repr(p))     # Point(3, 4)    - calls __repr__
print(p)           # (3, 4)          - uses __str__
```

Slide 12: Polymorphism with `isinstance()` and `issubclass()`

Checking polymorphic relationships at runtime:

```
class Animal: pass
class Dog(Animal): pass
class Cat(Animal): pass

d = Dog()
print(isinstance(d, Animal))  # True - Dog IS-A Animal
print(isinstance(d, Dog))     # True
print(isinstance(d, Cat))     # False

def describe(animal):
    if isinstance(animal, Dog):
        print("A loyal friend")
    elif isinstance(animal, Cat):
        print("An independent companion")
    else:
        print("Some animal")

describe(Dog())  # A loyal friend
describe(Cat())  # An independent companion
```

- Prefer duck typing over `isinstance()` when possible
- `isinstance()` is useful when behavior differs fundamentally by type

Slide 13: Polymorphism with `@property`

Properties enable polymorphic attribute access:

```
class Product:
    def __init__(self, base_price):
        self._base_price = base_price

    @property
    def final_price(self):
        return self._base_price

class DiscountedProduct(Product):
    def __init__(self, base_price, discount_pct):
        super().__init__(base_price)
```

```

        self.discount_pct = discount_pct

    @property
    def final_price(self):
        return self._base_price * (1 - self.discount_pct / 100)

class TaxedProduct(Product):
    def __init__(self, base_price, tax_rate):
        super().__init__(base_price)
        self.tax_rate = tax_rate

    @property
    def final_price(self):
        return self._base_price * (1 + self.tax_rate)

products = [Product(100), DiscountedProduct(100, 20), TaxedProduct(100, 0.1)]
for p in products:
    print(p.final_price)    # 100, 80, 110

```

Slide 14: Polymorphic Containers

Storing different types in a single collection:

```

class UIWidget:
    def render(self):
        pass

class Button(UIWidget):
    def __init__(self, label):
        self.label = label

    def render(self):
        print(f"[Button: {self.label}]")

class TextBox(UIWidget):
    def __init__(self, text):
        self.text = text

    def render(self):
        print(f"[TextBox: {self.text}]")

class Checkbox(UIWidget):
    def __init__(self, checked):
        self.checked = checked

    def render(self):
        status = "✓" if self.checked else "☐"
        print(f"[Checkbox: {status}]")

# Polymorphic container

```

```
widgets = [Button("Submit"), TextBox("Hello"), Checkbox(True)]
for w in widgets:
    w.render()
```

Slide 15: Method Overriding for Polymorphism

Key rules for method overriding:

```
class Base:
    def method(self):
        print("Base.method")

    def show(self, x):
        print(f"Base.show({x})")

class Derived(Base):
    def method(self):          # Override – same signature
        print("Derived.method")

    def show(self, x, y=None): # Extend – added optional param
        super().show(x)
        if y is not None:
            print(f"Extra: {y}")

d = Derived()
d.method()          # Derived.method – overridden
d.show(10)          # Base.show(10) – called via super()
d.show(10, 20)      # Base.show(10) then Extra: 20
```

Slide 16: Polymorphism with `__init_subclass__`

React to subclass creation automatically:

```
class PluginBase:
    registry = {}

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        PluginBase.registry[cls.__name__] = cls
        print(f"Registered plugin: {cls.__name__}")

    def execute(self):
        raise NotImplementedError

class PDFExporter(PluginBase):
    def execute(self):
        print("Exporting to PDF...")
```



```

class CSVExporter(PluginBase):
    def execute(self):
        print("Exporting to CSV...")

# Automatically registered!
print(PluginBase.registry)
# {'PDFExporter': <...>, 'CSVExporter': <...>}

# Polymorphic execution
for name, cls in PluginBase.registry.items():
    plugin = cls()
    plugin.execute()

```

Slide 17: Polymorphism with `functools.singledispatch`

Single-dispatch generic functions:

```

from functools import singledispatch

@singledispatch
def process(data):
    print(f"Default: {data}")

@process.register(int)
def _(data):
    print(f"Integer: {data * 2}")

@process.register(str)
def _(data):
    print(f"String: {data.upper()}")

@process.register(list)
def _(data):
    print(f"List: sum = {sum(data)}")

process(10)           # Integer: 20
process("hello")      # String: HELLO
process([1, 2, 3])    # List: sum = 6
process(3.14)         # Default: 3.14 (fallback)

```

- Dispatch based on the **first argument's type**
- No need to modify original classes — **open for extension**
- Registered implementations can be anywhere in the codebase

Slide 18: Polymorphism with Multiple Inheritance

Multiple inheritance + polymorphic dispatch:

```
class Flyable:
    def move(self):
        return "Flying in the air"

class Swimmable:
    def move(self):
        return "Swimming in water"

class Duck(Flyable, Swimmable):
    def move(self):
        return f"{Flyable.move(self)} and {Swimmable.move(self)}"

class Airplane(Flyable):
    pass

class Submarine(Swimmable):
    pass

# Polymorphic call
things = [Duck(), Airplane(), Submarine()]
for t in things:
    print(t.move())
# Flying in the air and Swimming in water
# Flying in the air
# Swimming in water
```

Slide 19: MRO and Polymorphism

MRO determines which method runs:

```
class A:
    def method(self):
        print("A.method")

class B(A):
    def method(self):
        print("B.method")

class C(A):
    def method(self):
        print("C.method")

class D(B, C):
    pass

d = D()
d.method()          # B.method – follows MRO
print(D.__mro__)
# D → B → C → A → object
```

```
# Polymorphism: treat D as any ancestor
for cls in [A, B, C]:
    obj = cls()
    obj.method()
# A.method, B.method, C.method
```

Slide 20: Polymorphism with Mixins

Composable behavior through multiple inheritance:

```
class JSONMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)

class XMLMixin:
    def to_xml(self):
        parts = [f"<{k}>{v}</{k}>" for k, v in self.__dict__.items()]
        return f"<{self.__class__.__name__}>{''.join(parts)}</{self.__class__.__name__}>"

class Person(JSONMixin, XMLMixin):
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"{self.name}, {self.age}"

p = Person("Alice", 30)
print(p.to_json())    # {"name": "Alice", "age": 30}
print(p.to_xml())     # <Person><name>Alice</name><age>30</age></Person>
```

- Mixins provide **horizontal** code reuse
- Different mixins offer different serialization formats
- Add/remove mixins without changing core class logic

Slide 21: The Template Method Pattern

Polymorphism to define algorithm structure:

```
from abc import ABC, abstractmethod

class DataProcessor(ABC):
    def process(self):
        data = self.read_data()
        # Template method
```

```

        transformed = self.transform(data)
        result = self.analyze(transformed)
        self.output(result)
        return result

    def read_data(self):
        return input("Enter data: ")          # Common step

    @abstractmethod
    def transform(self, data):
        pass                                  # Subclass defines

    @abstractmethod
    def analyze(self, data):
        pass                                  # Subclass defines

    def output(self, result):
        print(f"Result: {result}")           # Common step

class ReverseProcessor(DataProcessor):
    def transform(self, data):
        return data[::-1]

    def analyze(self, data):
        return f"Reversed: {data}"

class UpperProcessor(DataProcessor):
    def transform(self, data):
        return data.upper()

    def analyze(self, data):
        return f"Uppercase: {data}"

```

Slide 22: Polymorphism with Callables

Functions and classes as polymorphic callables:

```

# Any object with __call__ is a callable
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, value):
        return value * self.factor

class Adder:
    def __init__(self, increment):
        self.increment = increment

    def __call__(self, value):

```

```

        return value + self.increment

# Polymorphic callables
transformers = [Multiplier(2), Adder(10), Multiplier(3)]

value = 5
for t in transformers:
    value = t(value)
    print(value)
# 10 (5 * 2)
# 20 (10 + 10)
# 60 (20 * 3)

```

Functions, lambda, and classes with `__call__` are all interchangeable.

Slide 23: The Strategy Pattern

Polymorphism to swap algorithms at runtime:

```

from abc import ABC, abstractmethod

class SortStrategy(ABC):
    @abstractmethod
    def sort(self, data):
        pass

class BubbleSort(SortStrategy):
    def sort(self, data):
        print("Using bubble sort")
        return sorted(data) # Simplified

class QuickSort(SortStrategy):
    def sort(self, data):
        print("Using quick sort")
        return sorted(data) # Simplified

class MergeSort(SortStrategy):
    def sort(self, data):
        print("Using merge sort")
        return sorted(data) # Simplified

class Sorter:
    def __init__(self, strategy):
        self.strategy = strategy

    def sort(self, data):
        return self.strategy.sort(data)

# Swap strategies at runtime
data = [3, 1, 4, 1, 5]

```

```
sorter = Sorter(QuickSort())
print(sorter.sort(data))      # Using quick sort

sorter.strategy = MergeSort()
print(sorter.sort(data))      # Using merge sort
```

Slide 24: Polymorphism and Context Managers

Duck typing with context managers:

```
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        print(f"Opening {self.filename}")
        return self

    def __exit__(self, *args):
        print(f"Closing {self.filename}")

class DatabaseConnection:
    def __enter__(self):
        print("Connecting to database")
        return self

    def __exit__(self, *args):
        print("Disconnecting from database")

# Same `with` statement works for both
def use_resource(manager):
    with manager as res:
        print(f"Using {type(res).__name__}")

use_resource(FileManager("test.txt", "r"))
use_resource(DatabaseConnection())
```

Slide 25: Polymorphism with `__getattr__` and `__getattribute__`

Dynamic attribute delegation:

```
class Proxy:
    def __init__(self, target):
        self._target = target

    def __getattr__(self, name):
```

```

        # Called when normal lookup fails
        print(f"Delegating {name} to {type(self._target).__name__}")
        return getattr(self._target, name)

    def __call__(self, *args, **kwargs):
        print("Proxy called")
        return self._target(*args, **kwargs)

class Logger:
    def log(self, msg):
        print(f"LOG: {msg}")

proxy = Proxy(Logger())
proxy.log("test")    # Delegating log to Logger / LOG: test

# Proxy can wrap any object polymorphically
proxy2 = Proxy(lambda x: x * 2)
print(proxy2(5))    # Proxy called / 10

```

Slide 26: Common Pitfall — Forgetting Duck Typing Limits

Runtime errors from missing methods:

```

class Bird:
    def fly(self):
        print("Flying")

class Penguin:
    def swim(self):
        print("Swimming")

def make_it_fly(thing):
    thing.fly()    # Works for Bird, fails for Penguin

make_it_fly(Bird())    # Flying
# make_it_fly(Penguin()) # AttributeError: 'Penguin' has no attribute 'fly'

```

Solutions:

```

# 1. EAFP (Easier to Ask Forgiveness than Permission)
def make_it_fly(thing):
    try:
        thing.fly()
    except AttributeError:
        print("This thing can't fly")

# 2. hasattr() check
def make_it_fly(thing):

```

```
if hasattr(thing, 'fly'):
    thing.fly()
else:
    print("This thing can't fly")
```

3. ABC registration for formal interfaces

Slide 27: Common Pitfall — Accidental Method Override

Inheriting from third-party classes:

```
class Base:
    def save(self):
        print("Saving to database")

class Logger:
    def save(self):
        print("Logging data")

class User(Base, Logger):  # Order matters!
    pass

user = User()
user.save()  # "Saving to database" – Base wins (MRO)
```

```
# Better: avoid name collisions
class User(Base, LogMixin):
    def save(self):
        Base.save(self)  # Explicitly call Base's save
        self.log("Saved")  # Use mixin's log method
```

Always check MRO when combining classes with same method names.

Slide 28: Common Pitfall — Not Using `super()` Properly

Breaking the polymorphic chain:

```
class A:
    def __init__(self):
        print("A.__init__")

class B(A):
    def __init__(self):
        print("B.__init__")
        # Forgot: super().__init__()
```



```
class C(B):
    def __init__(self):
        print("C.__init__")
        super().__init__()

c = C()
# Output:
# C.__init__
# B.__init__
# (A.__init__ never called!)
```

A's constructor never runs because B didn't call `super().__init__()`. Always chain `super().__init__()` to maintain the MRO chain.

Slide 29: Polymorphism with `__instancecheck__` and `__subclasscheck__`

Customizing isinstance/issubclass behavior:

```
class AbstractMeta(type):
    def __instancecheck__(cls, instance):
        # Custom logic for isinstance()
        if hasattr(instance, 'quack'):
            return True
        return super().__instancecheck__(instance)

    def __subclasscheck__(cls, subclass):
        # Custom logic for issubclass()
        if hasattr(subclass, 'quack'):
            return True
        return super().__subclasscheck__(subclass)

class DuckLike(metaclass=AbstractMeta):
    pass

class Duck:
    def quack(self):
        pass

class Person:
    def quack(self):
        pass

print(isinstance(Duck(), DuckLike))    # True – has quack
print(isinstance(Person(), DuckLike))  # True – has quack
```

Slide 30: Composition over Inheritance

Polymorphism through delegation instead of inheritance:

```
# Instead of deep inheritance:
class EmailSender:
    def send(self, msg):
        print(f"Sending email: {msg}")

class SMSSender:
    def send(self, msg):
        print(f"Sending SMS: {msg}")

class NotificationService:
    def __init__(self, sender):
        self.sender = sender    # Composition: HAS-A sender

    def notify(self, msg):
        self.sender.send(msg)

# Polymorphic without inheritance
email_service = NotificationService(EmailSender())
sms_service = NotificationService(SMSSender())

email_service.notify("Hello")    # Sending email: Hello
sms_service.notify("Hello")      # Sending SMS: Hello
```

Favor composition: more flexible, looser coupling, easier to test.

Slide 31: Best Practices

Do's:

- ✓ Use duck typing for maximum flexibility
- ✓ Apply ABCs to define formal polymorphic interfaces
- ✓ Always call `super().__init__()` in constructors
- ✓ Prefer EAFP (try/except) over LBYL (hasattr) for duck typing
- ✓ Use `@singledispatch` for type-based dispatch on existing code
- ✓ Favor composition over inheritance for shared behavior

Don'ts:

- ✗ Overuse `isinstance()` checks — let duck typing work
 - ✗ Create deep inheritance hierarchies (3+ levels)
 - ✗ Forget to call `super()` in overridden methods
 - ✗ Assume all objects have a method — handle `AttributeError` gracefully
 - ✗ Use inheritance just to reuse one method (use composition)
-

Slide 32: Summary

Polymorphism in Python:

- **Duck Typing:** "If it walks like a duck..." — methods at runtime, not types
- **Method Overriding:** Child redefines parent method for different behavior
- **Operator Overloading:** `__add__`, `__str__`, etc. for custom operators
- **ABCs:** Formal interfaces for polymorphic contracts

Key Mechanisms:

- `super()` — chains overridden methods following MRO
- `@singledispatch` — type-based dispatch without modifying classes
- Mixins — composable, reusable behavior chunks
- `isinstance()/issubclass()` — runtime type checking

Core Principle: Write code that works with **interfaces and behaviors**, not concrete types. This makes your code flexible, extensible, and maintainable.

Slide 33: Resources & Further Reading

Official Documentation:

- Python's `abc` module: docs.python.org/3/library/abc.html
- `functools.singledispatch`: docs.python.org/3/library/functools.html
- Python data model (dunder methods): docs.python.org/3/reference/datamodel.html
- Duck typing Wikipedia: en.wikipedia.org/wiki/Duck_typing

Topics for Further Study:

- Protocols (structural subtyping) — PEP 544
 - Generic types and `TypeVar` for polymorphic functions
 - `__init_subclass__` for automatic registration patterns
 - Dependency injection for runtime strategy swapping
 - Adapter pattern — making incompatible types work together
-

Thank You!

Questions?